

Fit-Buddy: AI-Powered Fitness Planner

Project Description:

Fit-Buddy is an AI-powered fitness planning web application that generates personalized 7-day workout plans from user inputs such as name, user ID, age, weight, fitness goal, and workout intensity. It also provides nutrition and recovery guidance.

The application uses Google Gemini models for workout generation and nutrition tips. Users can submit feedback to refine an existing workout plan while preserving the original plan. SQLite with SQLAlchemy stores user and plan information for later review.

Scenarios

Scenario 1: A user enters their fitness details and selects a goal and workout intensity. Fit-Buddy generates a personalized 7-day workout plan.

Scenario 2: A user submits feedback on the generated plan. The system produces an updated plan based on the feedback while retaining the original plan.

Scenario 3: A user receives concise nutrition/recovery guidance generated alongside the workout planning workflow.

Technical Architecture

Description:

Fit-Buddy uses a modular web architecture. Jinja2 templates provide the frontend interface, FastAPI handles routing, validation and application logic, Google Gemini provides generative AI responses, and SQLite with SQLAlchemy provides persistent storage.

Main flow: User -> Jinja2 page -> FastAPI route -> validation/business logic -> Gemini -> SQLite/SQLAlchemy -> result/dashboard.

Pre-requisites:

- Python programming and virtual-environment setup.
- FastAPI and Uvicorn fundamentals.
- HTML, CSS, JavaScript and Jinja2 template knowledge.
- SQLite and SQLAlchemy database basics.
- Google Gemini API configuration and secure environment-variable handling.
- Git/version-control familiarity.

Project Workflow

Milestone 1: Model Selection and Architecture

- Research and select the Gemini model used for workout generation, update logic and nutrition guidance.
- Define the interaction between Jinja2 frontend, FastAPI backend, Gemini services and SQLite database.
- Set up the Python development environment and dependencies.

Milestone 2: Core Functionalities Development

- Implement personalized 7-day workout generation.
- Implement nutrition/recovery tip generation and feedback-driven plan updates.
- Implement user and plan persistence.

Milestone 3: Backend Development

- Implement FastAPI routes, validation, business logic and database access.
- Expose /generate-workout and /submit-feedback workflows.

Milestone 4: Frontend Development

- Develop dynamic Jinja2 pages for input, generated results and the coach/admin view.

Milestone 5: Local Deployment and Testing

- Run the FastAPI application locally with Uvicorn and verify the main workflows.

Milestone 6: Conclusion

Complete integration, testing, documentation and local deployment verification.

Milestone 1: Model Selection and Architecture

Activity 1.1: Configure the Gemini API

Configure the Google Gemini API key through environment variables. The API key is kept outside source code and supplied through the project environment.

Activity 1.2: Select the Generative AI Model

Fit-Buddy uses Gemini 1.5 Pro for workout generation and plan updates, with Gemini 1.5 Flash used for concise nutrition/recovery tips.

Activity 1.3: Define the Application Architecture

Define the flow between Jinja2 templates, FastAPI routes, validation/business logic, Gemini generation functions and SQLite/SQLAlchemy persistence.

the Gemini API

Secure API Configuration

Create a local environment file for the Gemini API key and keep the key out of committed source files. Use the project's environment configuration during local execution.

Application Integration

The backend invokes the Gemini generation functions when a valid workout-generation request is submitted. The generated response is then prepared for display and persistence.

Security Practice

- Do not hard-code the API key in application source code.
- Keep environment configuration separate from application logic.
- Use the provided dependency and setup instructions when preparing the local environment.

odel

Selection Requirements

- Generate structured, personalized 7-day workout plans from user fitness inputs.
- Support plan refinement from user feedback.
- Provide concise nutrition/recovery guidance.

Selected Models

Gemini 1.5 Pro is used for workout generation and update logic. Gemini 1.5 Flash is used for nutrition/recovery tip generation.

Activity 1.3: Define the Architecture

- Frontend: HTML/CSS/JavaScript with Jinja2 templates.
- Backend: Python, FastAPI and Uvicorn.
- AI layer: Gemini generation functions.
- Persistence: SQLite with SQLAlchemy.
- Administrative view: registered users and stored plans.

Activity 1.4: Set Up the Development Environment

Environment Setup

- Install Python and create a virtual environment.
- Install the dependencies listed in fitbuddy/requirements.txt.
- Configure the Gemini API key through the environment configuration.
- Keep the project source, templates and test file in the supplied project structure.

Project Structure

The application uses an app package containing main.py, routes.py, database.py, schemas.py, Gemini generator modules and updated_plan.py, with Jinja2 templates and test_api.py separated into their respective locations.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Fitness Features

- Personalized 7-day workout plan generation.
- Nutrition/recovery tip generation.
- Feedback-driven workout plan update.
- User and plan persistence in SQLite.
- Coach/admin view of registered users and stored plans.

Activity 2.2: Implement the FastAPI Backend

- Define routes for the main application workflows.
- Validate request data using Pydantic schemas.
- Process inputs and invoke the appropriate Gemini generator.
- Store and retrieve users and plans using SQLite/SQLAlchemy.

Milestone 3: Main Application Logic

Activity 3.1: Backend Application Logic

main.py starts the FastAPI application and provides the application entry point. routes.py handles the web/API workflows and connects validated requests to the required business logic.

Main Routes

- / - main Fit-Buddy input page.
- /generate-workout - workout generation workflow.
- /submit-feedback - feedback-driven plan update workflow.
- /view-all-users - coach/admin view of stored users and plans.
- /docs - FastAPI interactive API documentation.

Request Processing

User input is validated, the relevant Gemini function is called, the generated result is prepared for the response, and applicable user/plan information is persisted in SQLite.

Workout Generation and Update Logic

Workout Generation

`generate_workout_gemini()` uses the user's fitness inputs to request a personalized 7-day workout plan. The generated plan is returned through the application workflow and stored for later review.

Feedback Update

`update_workout_plan()` uses the existing plan together with user feedback to generate a revised plan. The workflow preserves the original plan rather than replacing it.

Nutrition Guidance

`generate_nutrition_tip_with_flash()` uses Gemini 1.5 Flash to generate concise nutrition/recovery guidance associated with the user's fitness planning request.

Structured Input

Pydantic schemas are used to represent and validate application inputs before the core workflow processes them.

Fitness Inputs

- Name and user ID.
- Age and weight.
- Fitness goal.
- Workout intensity.

AI Prompt Construction

The backend passes validated user information and, where applicable, previous plan and feedback information to the Gemini generation functions so that the returned content is tailored to the requested fitness workflow.

AI Integration

The Gemini generator modules encapsulate calls to the configured Google Gemini models. Separate functions are used for workout generation, nutrition/recovery guidance and feedback-based plan updates.

Response Handling

- Receive the model-generated workout or guidance.
- Prepare the generated content for the application response.
- Persist applicable plan information in SQLite.
- Render the result through the Jinja2 templates.

API Documentation

FastAPI exposes the /docs interface so the available application endpoints can be reviewed and tested during development.

Milestone 4: Frontend Development

Activity 4.1: Designing and Developing the User Interface

The frontend uses HTML/CSS/JavaScript with Jinja2 templates to provide the Fit-Buddy input form and result pages.

Dynamic Templates

- index.html provides the user input workflow.
- result.html displays the generated fitness plan and guidance.
- all_users.html provides the coach/admin view of stored users and plans.

Activity 4.2: Dynamic Result Rendering

The FastAPI backend supplies generated content to the Jinja2 templates so that user-specific workout and guidance information is displayed dynamically.

Result Page

After workout generation, the application presents the personalized 7-day plan and associated nutrition/recovery guidance through the result template.

Feedback Workflow

The user can provide feedback on the generated plan. The feedback is submitted to the backend, which generates an updated plan while retaining the original.

Coach/Admin View

The all_users.html page provides a simple view of registered users and stored plans for review.

Milestone 5: Local Deployment

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a Python virtual environment.
- Install dependencies from fitbuddy/requirements.txt.
- Configure the Gemini API key through the environment file.
- Ensure the SQLite database and application modules are available.

Application Server

Run the FastAPI application with Uvicorn using the project's configured application entry point.

Local Testing and Verification

Activity 5.2: Test the Application Locally

- Start the FastAPI application with Uvicorn.
- Open the main page in a browser.
- Submit valid fitness information and generate a 7-day plan.
- Submit feedback and verify that an updated plan is produced.
- Check the nutrition/recovery guidance.
- Review stored users and plans through the coach/admin view.
- Use /docs to inspect and exercise the API endpoints.

Test Support

The project includes `test_api.py` for API-level testing of the application workflows.

Deployment / Access Details

Local Deployment

The supplied project is prepared for local execution. A public hosted application URL is not established in the supplied Fit-Buddy project information.

Repository

<https://github.com/lingeshwaranss/Fit-Buddy.git>

Demo Video

<https://drive.google.com/drive/folders/1kkW0nGt5YK2zUHWrvzS4sgOxkZuAa1Kc>

Environment Configuration

Use the project's environment configuration for the Gemini API key. Do not place secrets directly in source code.

Important Notes

- The application depends on a valid Google Gemini API configuration for AI-generated content.
- Local execution requires the project dependencies and Python environment to be configured.
- AI-generated workout and nutrition guidance should be treated as application output and reviewed appropriately by the user.
- No unsupported performance benchmark values are stated in this documentation.
- A hosted production URL or login credentials are not established by the supplied Fit-Buddy project information.

Exploring the Website's Web Pages

Home / Generator Page:

The main Fit-Buddy page collects the user's name, user ID, age, weight, fitness goal and workout intensity before submitting the workout-generation request.

Input Workflow

The user provides the required fitness information and selects the desired workout intensity. The form sends the request to the FastAPI backend for validation and AI processing.

Input and Results Sections

Input Section:

The input interface contains fields for Name, User ID, Age, Weight, Fitness Goal and Workout Intensity, with a Generate Plan action.

Results Section:

The result page presents the generated 7-day workout plan and nutrition/recovery guidance. Feedback can be submitted to request an updated plan while preserving the original plan.

Conclusion

Fit-Buddy is a generative AI fitness planning web application built with Python, FastAPI, Uvicorn, Jinja2, SQLite/SQLAlchemy and Google Gemini models. It generates personalized 7-day workout plans, provides nutrition/recovery guidance, and supports feedback-driven plan refinement.

The project combines AI generation with persistent user and plan storage and provides a coach/admin view for reviewing stored information. The implementation also includes API documentation and API-level testing support.

Team

- Lingeshwaran S S - Team Leader; UI/design, development environment setup and conclusion.
- Prathish R - project workflow, core functionality and dynamic templates.
- Akash S - Gemini model research/selection and FastAPI input processing.
- Dinesh S - architecture, main application logic and local deployment testing.